

Build prompt: location-based event discovery & verified-ticketing app

Use this document as the complete specification. Follow every section in order. Do not skip a section, do not silently drop a requirement — if something is ambiguous, state your assumption explicitly in your response and proceed with the most conservative safe choice.

1. Product summary

Build a mobile-first app where users discover nearby events on a live map, buy tickets, and check in at the door using a unique, cryptographically verifiable ticket — one that cannot be duplicated, screenshotted-and-reused, or oversold. The app also supports safe peer-to-peer resale and a lightweight social "going" layer.

Explicit non-goal: do not attempt to support large arena/stadium-scale exclusive promoter ticketing (that market is locked by exclusive contracts). Optimize for small-to-mid venues: bars, clubs, pop-ups, community events, local shows.

2. Scale & non-functional targets

Design for these numbers even in an MVP — do not hardcode assumptions that break past them:

Baseline: tens of thousands of monthly active users, thousands of active listings, mostly read-heavy location browsing.

Purchase spikes: a single popular event must support 5,000–20,000 concurrent purchase attempts within a 30-second window without overselling or crashing.

Event-night check-in: hundreds to low-thousands of check-ins within a 1–2 hour window at one venue, on venue wifi that may degrade or drop entirely.

Target purchase flow latency: sub-second perceived response on "buy" (optimistic UI allowed, see §7).

Availability target: discovery/browsing degrades gracefully (cached/stale results) rather than hard-failing if backend services are degraded; purchase and check-in paths must fail safe (never oversell, never let two people in on one duplicated ticket).

3. System architecture

3.1 Clients

Attendee mobile app (iOS/Android) — browse, buy, hold tickets, resale, "going" status.

Organizer web dashboard — create/manage events, view real-time sales, check-in analytics, refunds.

Venue scanner mode — same mobile app, staff-facing scan screen; must work with intermittent connectivity.

3.2 Backend services (logically separate; co-locate on one deploy for MVP if needed, but keep clean boundaries so they can be split later)

Service	Responsibility
Listings/Events	CRUD for events; geo-tagged; category/time metadata. Source of truth: Postgres + PostGIS.
Search/Discovery	Serves "events near me" queries; radius/category/time filters; caches hot queries.
Ticketing/Inventory	Owns per-event ticket counts; enforces no-oversell purchase logic (see §6).
Payments	Wraps a PCI-compliant processor (e.g. Stripe). Never store raw card data. Idempotent charge handling + webhook reconciliation.
Ticket Verification	Issues unique signed per-ticket credentials; validates/marks-used at the door; handles resale reissue (see §6, §8).
Social/Going	Tracks opt-in attendance and friend graph; feeds Discovery (social proof) and Notification. New service — do not bolt onto Ticketing.
Notification	Push/SMS/email for purchase confirmation, reminders, gate updates, friend-attendance alerts. Decoupled via queue so it never blocks purchase or check-in.

3.3 Data stores

Postgres (with PostGIS extension) — events, users, tickets, orders: source of truth.

Redis — hot query caching, purchase-flow rate limiting, distributed locks/atomic counters for inventory.

Object storage (S3-compatible) — event images/media.

Queue (SQS/Kafka/equivalent) — decouples purchase-confirmed → ticket-issued → notification-sent so a slow downstream (e.g. push provider) never blocks a purchase or check-in.

4. Data model requirements (minimum viable schema — extend as needed, do not shrink)

`events`: id, organizer_id, title, description, geo point (lat/lng, indexed), venue_address, start_time, end_time, category, capacity, status.

`tickets`: id, event_id, owner_user_id, order_id, status (`valid`, `used`, `revoked`, `refunded`), signed_token, created_at, used_at.

`orders`: id, buyer_user_id, event_id, quantity, payment_intent_id, status, created_at.

`users`: id, auth fields, profile, notification preferences.

`going`: user_id, event_id, visibility (opt-in per user; default private).

`resale_transfers`: id, original_ticket_id, new_ticket_id, seller_id, buyer_id, price, status, created_at.

5. Hard problems — required solutions (do not simplify away)

Oversell prevention under spike load. Purchases must be atomic against remaining inventory. Use a database-level atomic decrement (e.g. `UPDATE events SET tickets_remaining = tickets_remaining - N WHERE tickets_remaining >= N` checked via affected-row-count) or a Redis-backed atomic counter reconciled against Postgres as source of truth. Queue/rate-limit incoming purchase requests ahead of the hot path so the DB is never hit with unbounded concurrency. Never allow a "reserve then confirm" gap longer than a few seconds without an expiring hold.

Geospatial search at scale. Use PostGIS spatial indexes (GiST) for radius queries; cache popular city/area queries in Redis with short TTL; paginate results; do not do full-table scans for "near me."

Ticket verification without reliable connectivity. See §6 — this is the most novel and highest-risk piece. Build it deliberately, not as an afterthought.

Payment correctness. All charge operations must be idempotent (idempotency keys on create-order). Reconcile via payment provider webhooks, not just the synchronous response. Handle partial failures (charge succeeded, ticket issuance failed) with a retry/compensation path — never leave a paid user without a ticket, and never issue a ticket without confirmed payment.

6. Ticket verification design (build this exactly as specified)

Build a **hybrid** system:

Default path — online atomic validation. Each ticket has a unique ID plus a signed token (server-issued signature over ticket ID + event ID + expiry). At the door, the scanner calls a verify endpoint that atomically checks-and-marks the ticket `used` in a single transaction (e.g. `UPDATE tickets SET status='used' WHERE id=? AND status='valid'` and check affected rows = 1). This guarantees the same ticket cannot be scanned twice at two different doors.

Offline fallback — only when connectivity is confirmed lost. Scanner devices pre-sync a compact used-ticket set (bloom filter or hash set) for their assigned event before doors open. Offline, the scanner verifies the token's signature locally (no network call) and checks the local used-set, then queues the scan for async sync when connectivity returns.

Explicitly handle the known race: two offline scanners could let the same ticket in before syncing. Do not pretend this can't happen. Implement a reconciliation job that flags any ticket scanned more than once across devices as a fraud-review event for staff — this is accepted as an after-the-fact detection, not a real-time prevention, and must be documented as such in the UI for staff (e.g. "flagged: possible duplicate entry").

Both paths must use the same signing/token format so resale reissue (§8) works uniformly.

7. Purchase flow UX (engaging + fast)

One-tap buy with saved payment method.

Optimistic UI: show "you're in!" confirmation immediately on submit, with the real server confirmation resolving just behind it; if the server ultimately fails (sold out in the race), show an honest, non-alarming failure state and offer a waitlist join — never a raw error page.

Waitlist: if sold out, let users join a queue and get notified if inventory frees up (cancellations/refunds).

Ticket delivery: instant in-app ticket with QR/token visible even without connectivity (cache it client-side after issuance).

8. Resale flow (build using the existing verification primitive — do not build new fraud logic)

Seller requests resale on an owned, valid ticket.

Ticket Verification service **revokes** the original signed token (mark `revoked`; it can never validate again).

Ticket Verification service **issues a new signed token** bound to the buyer's user ID, referencing the same event.

Buyer is notified via the Notification service and receives a fully valid new ticket.

Record the transfer in `resale_transfers` for audit/analytics. Enforce organizer-configurable resale rules (allowed/disallowed, price cap) at this step.

9. Social "going" layer (privacy-first — do not default this to public)

Attendance visibility is **opt-in and default-private**. A user must explicitly choose to share that they're going, and to whom (e.g. friends-only vs public).

Social/Going service feeds:

Discovery — boosts/annotates events with "N friends going" only for users who've opted in and only showing friends who also opted in.

Notification — optional alert when a friend marks themselves going to an event the user is also viewing/considering.

This is a distinct service from Ticketing — do not store attendance-sharing preferences on the ticket or order record.

10. UI/UX requirements — what makes this engaging, not just functional

Implement all of the following; they are differentiators against incumbents, not optional polish:

Map-first discovery as the default home view (not a list) — pins reflect "happening soon," filterable by vibe/category, real-time-feeling.

Social proof ("going") signal on event cards and detail pages, respecting §9 privacy rules.

Frictionless resale surfaced directly on the ticket screen ("Can't make it? Resell instantly") — not buried in settings.

Organizer dashboard quality bar: real-time sales chart, one-tap refund, live check-in rate/line-speed view. This must feel materially better than a generic admin CRUD screen.

Sub-second checkout per §7.

Post-event loop: after an event ends, prompt the attendee for a quick photo/reaction, feeding back into that event's/organizer's future discovery listings (with consent).

11. Security & privacy requirements

Never store raw payment card data — delegate to the payment processor's vault/tokenization.

Ticket tokens must be cryptographically signed (asymmetric signing preferred, e.g. Ed25519) so scanners can verify offline without holding a secret that could be extracted from a device and used to forge tickets.

Attendance/"going" data is sensitive — treat as private-by-default per §9; do not expose it in any public API without the sharing user's explicit consent.

Rate-limit purchase and verification endpoints to blunt scalping bots and brute-force scan attempts.

12. Acceptance criteria (the agent must self-check against these before declaring done)

- ☐ Cannot oversell an event under simulated concurrent load (write and run a load test proving this).
- ☐ Cannot scan the same ticket twice for entry in the online path (atomic check enforced at the DB level, not just application logic).
- ☐ Offline scan path works with zero network connectivity and reconciles afterward; duplicate offline scans are flagged, not silently allowed.
- ☐ Resale reissues a token and permanently invalidates the old one — old QR must fail verification after resale.
- ☐ "Going" status is private by default; no attendance data is ever returned by any API to a user who hasn't been granted visibility.
- ☐ Payment failures never result in an issued ticket without confirmed payment, and never result in a charged user without a ticket (compensation/retry path exists and is tested).
- ☐ Map-first discovery view is the default landing screen and returns results within acceptable latency for a radius query against a large dataset (test at simulated hundreds-of-thousands-of-listings scale).

13. Assumptions and interpretations made while writing this prompt

These were not explicitly stated in the original discussion and were filled in conservatively — flag if any should change:

Assumed Postgres + PostGIS as the geo/relational store (vs. a dedicated search engine like Elasticsearch) since it keeps a single source of truth simpler for an MVP; noted Elasticsearch/OpenSearch as an upgrade path if search complexity grows.

Assumed Stripe-equivalent as the payment processor without naming a specific vendor — pick one PCI-compliant provider and use it consistently.

Assumed Ed25519 (asymmetric signing) for ticket tokens specifically so offline scanners never hold a secret capable of forging new tickets — this is stricter than a plain HMAC and should not be relaxed.

Assumed "hybrid" verification (online default, offline fallback only when connectivity is confirmed lost) rather than pure offline-first, per the earlier discussion's conclusion that this balances consistency and resilience.

Added the waitlist mechanic (§7) and post-event photo loop (§10.6) as concrete implementations of ideas raised more generally earlier ("honest sold-out state," "post-event loop") — implement as described unless you have a stronger alternative, in which case state it explicitly rather than silently substituting.